



School of Future Tech

Case Study Report

on

LedgerAudit: High-Performance Financial Analysis Platform

by

Ankitraj Jha

150096725050

GitHub Repository: <https://github.com/Ankitraj-17/LedgerAudit>

Live Application Demo: <https://ledger-audit.vercel.app/>

Index

1. Introduction to the Case Study.
2. Problem Statement / Case Background (Abstract).
3. Problem Statement / Case Study Design.
4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study.
5. Problem Statement / Case Study Implementation Details and Snapshots.
6. Problem Statement / Case Study Results and Conclusion.
7. References

1. Introduction to the Case Study

The digitization of global finance has produced an environment of unprecedented complexity. Multi-billion-dollar enterprises route funds across dozens of international banking institutions, clearing houses, and offshore intermediaries on a daily basis. For a forensic accountant or compliance officer attempting to audit these operations, the sheer volume and topological complexity of ledger data have rendered traditional tools—static spreadsheets, legacy ERP dashboards, and manual review pipelines—entirely inadequate.

Modern auditing demands a new class of software: one that does not merely display data, but actively participates in the analytical process by algorithmically prioritizing risk, enforcing chronological review discipline, and visualizing complex financial networks in real time.

LedgerAudit is a next-generation financial forensic dashboard built as a Single Page Application (SPA) using React 19 and Vite. It is designed around a core architectural philosophy: every significant operational bottleneck in a real-world compliance workflow can be directly mapped to, and solved by, a specific computer science data structure. By deeply embedding Max Heaps, Graphs with BFS, LIFO Stacks, FIFO Queues, Hash Maps, and Singly Linked Lists into the React component layer, LedgerAudit transforms theoretical algorithms into tangible, high-performance forensic tools.

Core Application Features:

- **Landing Page Overview:** Introduces the LedgerAudit brand identity with a premium dark-mode interface, key system metric cards, and direct access to the auditor workspace.
- **Dashboard Overview:** Renders high-level KPI cards (Total Audited Volume, Active Anomalies, Flagged Items, Compliance Rate) alongside a live transaction activity feed and quick navigation grid, providing a central hub for audit monitoring.
- **Transaction History (LinkedList + Undo Stack):** Renders a filterable, paginated ledger data table powered by a Singly Linked List for sequential data ingestion and

traversal. Supports in-line status updates and immediate step-by-step state reversion managed via a LIFO Undo Stack.

- **Flagged Transaction Queue (FIFO):** Enforces strict regulatory compliance by sequencing suspicious records in a FIFO Queue, forcing auditors to resolve transactions in their exact chronological order with no selective skipping or cherry-picking.
- **Expense Risk Sorter (Max Heap):** Automatically aggregates and ranks ledger transactions by budget deviation percentage using a custom Max Heap. It surfaces the most significant overruns to the top of the list in $O(\log n)$ insertion/extraction time.
- **Money Movement Network Map & BFS Path Finder:** Models financial wire routes as a directed, sparse Graph stored as an Adjacency List. Features an interactive SVG visualization and a Breadth-First Search (BFS) shortest-path query engine that traces multi-hop fund flows, calculating total hop counts and wire values.
- **Task Assignor (Greedy Algorithm):** Automates workload distribution to forensic specialists using a Greedy load-balancing algorithm. It matches transaction categories with auditor domain keywords, penalizes scores based on caseload and busy status, and falls back to minimum-load assignment.
- **Transaction Code Lookup (HashMap):** Uses a JavaScript Map-based directory to perform $O(1)$ constant-time searches for unique transaction IDs, instantly displaying verification status, authorized amounts, and detailed compliance notes.
- **Profile, Pricing & Subscription Management:** Manages auditor user context, active subscription tiers (Standard, Professional, Enterprise), billing intervals (monthly/annually), and system dark/light theme options, persisting all settings in localStorage.

2. Problem Statement

The technical challenges are:

- **Computational Sorting Bottleneck:** Standard browser-side sort operations applied to growing financial ledgers cause main-thread blocking and UI freezes, degrading the auditor's efficiency as the dataset scales.
- **State Reversion Vulnerability:** Manual edits by auditors can permanently overwrite original ledger data in poorly designed systems, destroying the forensic trail and making reliable undo operations critical.
- **Auditor Bias in Compliance Workflows:** When presented with a list of flagged transactions, auditors tend to cherry-pick simpler items, violating strict compliance chronologies and creating regulatory exposure.
- **Opacity of Layered Money Movement:** Financial fraud is rarely a point-to-point transfer. Funds are routed through complex multi-hop networks of intermediary institutions—a topology entirely invisible in standard tabular data views.

- **Static State Across Sessions:** User preferences, active subscriptions, and audit session context are lost on browser refresh, forcing auditors to reconfigure their environment repeatedly.

Case Study Objectives:

This case study aims to address these challenges by focusing on three primary objectives:

- **Algorithm-Driven Architecture:** Designing a system where each major UI module is directly powered by a specific, optimized data structure—Heap, Stack, Queue, Graph—rather than general-purpose array methods.
- **Zero-Latency Forensic Tools:** Implementing client-side, in-memory algorithmic processing to guarantee sub-millisecond response times for all core analytical operations.
- **Persistent, Tamper-Resistant Audit Environment:** Deploying a Cryptographic Vault Seal write-protection mechanism and robust localStorage synchronization to protect audit integrity across browser sessions.

3. Case Study Design

The system employs a Strict Separation of Concerns Architecture with Unidirectional Data Flow.

System Overview

A. Inputs

The system receives input from three primary sources:

- **User Interactions:** Auditor actions such as flagging transactions, editing ledger entries, and querying the network map are captured as discrete React events and dispatched through the centralized state hook.
- **Seed Data Ingestion:** A structured mock ledger dataset containing 12 high-fidelity financial transactions—including approved, flagged, suspicious, and mismatched records—is loaded at application startup.
- **Session Hydration:** Theme preferences, user identity, subscription plan, and billing interval are automatically restored from localStorage when the application mounts.

B. Architectural Constraints and Quality Attributes

The design adheres to the following critical constraints:

- **Immutable State:** All state mutations flow exclusively through React's setter functions, guaranteeing predictable, traceable re-renders with no direct data mutation.
- **Write Protection:** A global Vault Seal toggle locks the entire application interface, preventing any state mutation during the final report generation phase.
- **Single Source of Truth:** All business logic, global state, and cross-module data are centralized within the custom useAppState hook, eliminating prop-drilling and state fragmentation across components.

C. Conceptual Framework

The project is modeled as a Forensic Investigation Pipeline. Each transaction in the ledger cycles through a defined lifecycle:

INGESTED → UNDER REVIEW → FLAGGED → ASSIGNED → RESOLVED

The data structure layer acts as the engine for each transition: the Heap surfaces flagged items by risk, the Queue enforces review order, and the Stack preserves prior state for safe reversion.

D. Core Technology Stack

The underlying technologies were selected for specific functional benefits:

- **React 19:** Chosen for its Concurrent Rendering capability, which is critical for preventing heavy algorithmic calculations such as heap insertions and BFS traversals from interrupting smooth UI animations and user interactions.
- **Vite:** Selected as the build tool for its near-instantaneous Hot Module Replacement during development and its highly optimized, tree-shaken production bundles that minimize load times.
- **React Router DOM v7 using HashRouter:** Implements hash-based routing which eliminates 404 errors on static hosting platforms like Vercel without requiring any server-side route configuration.
- **Tailwind CSS v4 with CSS Variables:** A utility-first styling approach augmented with semantic CSS variable tokens enables a fully functional light/dark theme system without any external theming library, switching the entire UI palette by updating a single HTML attribute.
- **Lucide-React:** Provides crisp, scalable, consistent SVG vector icons across the navigation, status indicators, and action buttons throughout the application.

E. Component Architecture

LedgerAudit employs a strict Separation of Concerns Architecture where each source file has a dedicated role:

- **Entry Point (src/main.jsx):** Bootstraps the application inside a React Concurrent Root, wrapping the App component in a HashRouter to handle client-side routing on static hosting environments.
- **Root Component (src/App.jsx):** Acts as a logic-free layout shell that delegates all page-level routing and UI layout wrapping to the AppRoutes component.
- **Routing & Layout Layer (src/routes/AppRoutes.jsx):** Sets up route definitions and path mappings for the application. Wraps active page views in shared layout components including Sidebar and TopNav.
- **Central State hook (src/hooks/useAppState.js):** Represents the single source of truth for global state, managing the core state values (ledger transaction list, active auditor profile, specialists directory, theme preference, Undo stack, and Vault Seal switch) and exposing state modifiers.
- **Pages (src/pages/):** The view layer composed of distinct workspaces: Landing.jsx, Overview.jsx, TransactionHistory.jsx, FlaggedQueue.jsx, RiskSorter.jsx, MoneyMovement.jsx, TaskAssignor.jsx, CodeLookup.jsx, Profile.jsx, and Pricing.jsx.

- **Reusable UI Components (src/components/):** Modular interface blocks such as LoadingSpinner.jsx, Overview.jsx (reusable summary cards), PricingModal.jsx, Sidebar.jsx, SkeletonBlock.jsx, and TopNav.jsx.
- **Algorithmic Utilities (src/utills/):** Houses pure JavaScript implementation classes for core data structures (MaxHeap.js, Graph.js, Stack.js, Queue.js, HashMap.js, LinkedList.js) and standard mock dataset generation (mockData.js).

4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study

LedgerAudit: Technical Architecture Overview

LedgerAudit is engineered on a "Local-First" methodology, meaning all intensive computational tasks—sorting, pathfinding, state reversion—are executed directly on the client-side for zero-latency performance.

I. Data Structure-Specific Core Algorithms

This section details the specialized algorithmic logic powering each analytical module:

- **Max Heap (Risk Sorter):** Transactions are inserted into a Max Heap keyed by a budget deviation percentage formula. For any transaction, the deviation is calculated as the difference between the actual amount and the budgeted amount, divided by the budgeted amount, expressed as a percentage. The heapifyUp and heapifyDown operations maintain the heap property at every insertion and extraction, guaranteeing $O(\log n)$ time complexity. This ensures the highest-risk anomaly is always available at the root of the heap without ever performing a full array re-sort.
- **Graph with Adjacency List (Money Movement Network):** Each financial institution is modeled as a vertex in a directed, weighted Graph. The graph is stored internally as an adjacency list using a JavaScript Map, where each key is an institution name and each value is an array of outgoing edge objects carrying the destination node, the transfer amount, and the currency. This structure is chosen over an adjacency matrix because financial networks are sparse—not every bank connects directly to every other—so the adjacency list consumes far less memory and iterates only over real connections. The graph is built by calling addEdge for each wire transfer found in the transaction dataset, which automatically creates both endpoints as nodes if they do not yet exist.
- **Breadth-First Search (BFS) — Quickest Path Finder:** BFS is implemented as a dedicated forensic query tool separate from the network map visualization. When an auditor selects a source institution and a target institution and triggers the path query, the BFS algorithm begins at the source node and explores the graph level by level, radiating outward one hop at a time. Rather than storing individual node identifiers in its queue, the implementation enqueues entire path arrays, so each dequeue operation retrieves the full chain of institutions visited from the source to the current node.
A visited Set prevents any institution from being explored twice, which eliminates infinite loops caused by circular transfer routes. The first time the target institution is reached, the algorithm immediately terminates and returns the complete shortest

path by hop count. It simultaneously computes the total monetary value transferred along that path by summing the edge weights between each consecutive pair of institutions in the result.

The output the path array, hop count, total wire value, and the order in which all nodes were visited during the search—is displayed in a dedicated result panel and used to drive the animated laser-flow trace on the SVG map.

- **LIFO Stack (Undo Log):** Every manual edit made by an auditor results in the previous full transaction state being packaged into an object and pushed onto a Stack. When the auditor clicks Undo, the system pops the top object off the stack and restores that exact state in $O(1)$ constant time. This provides a mathematically guaranteed, zero-latency reversion mechanism with no risk of data corruption.
- **FIFO Queue (Flagged Review Pipeline):** Suspicious transactions are enqueued in strict chronological order as they are flagged. The "Resolve Next" interface exposes only the dequeue operation, which retrieves and removes the oldest item from the front. This architecture physically prevents auditors from accessing items out of order, enforcing the regulatory review sequence at the data structure level.
- **Hash Map (Transaction Code Lookup):** An official code directory maps each unique transaction ID to its audit status (VALID, MISMATCH, or SUSPICIOUS), the authorized amount, and a detailed audit note. This enables $O(1)$ constant-time lookup for instant cross-referencing of any ledger entry against the official record, with no iteration over the dataset required.
- **Singly Linked List (Transaction History Log and Routing Path Traversal):** The Singly Linked List serves two distinct roles in LedgerAudit. First, it underpins the Transaction History Log: when the ledger dataset is loaded at application startup, transaction entries are appended node-by-node to a Singly Linked List using the append operation, building an ordered chain where each node holds a complete transaction record and a pointer to the next entry. The `toArray()` traversal then converts this chain into the array rendered by the history table, preserving the original insertion order across the full dataset. Second, when an auditor selects a specific transaction for network analysis, its `routingPath` array—the sequence of banking institutions through which the wire was routed—is loaded into a fresh Singly Linked List. The sequential traversal from head to tail maps each institution node in order, driving the step-by-step visual animation of funds flowing across the SVG topology map.

II. Data Management and Persistence

Strategies for maintaining state and navigation:

- **useAppState Custom Hook:** Acts as the single source of truth for all application data including the transactions array, specialists roster, undo log, vault sealed flag, current user, current plan, billing interval, and theme. No external state library is required.
- **localStorage Synchronization:** Theme, current user, subscription plan, and billing interval are persisted using lazy-initialized state reads and synchronized via

useEffect hooks. This guarantees that the auditor's session context survives full browser refreshes, simulating a desktop application experience.

- **Vault Seal:** A boolean flag broadcast globally from the central state hook disables all write operations across every module simultaneously when activated, providing a system-wide tamper lock for final report generation.

5. Problem Statement / Case Study Implementation Details and Snapshots.

1. Max Heap: The Risk Sorter Engine

- Located in the MaxHeap utility file, this class is the core algorithmic engine of the Risk Sorter module. It calculates a budget deviation score for each transaction by computing the percentage difference between the actual amount spent and the authorized budget. The heapifyUp method compares the newly inserted item against its parent node and repeatedly swaps them upward until the parent's deviation is greater, maintaining the max-heap property throughout.
- The complementary heapifyDown method is called after extraction to restore heap order from the root downward. The heapify method builds the entire heap from an existing array in linear $O(n)$ time at component mount, so subsequent individual insertions each cost only $O(\log n)$. This means the Risk Sorter view remains fully responsive and zero-latency regardless of how large the ledger grows.

2. BFS Shortest Path Tracer (Money Movement)

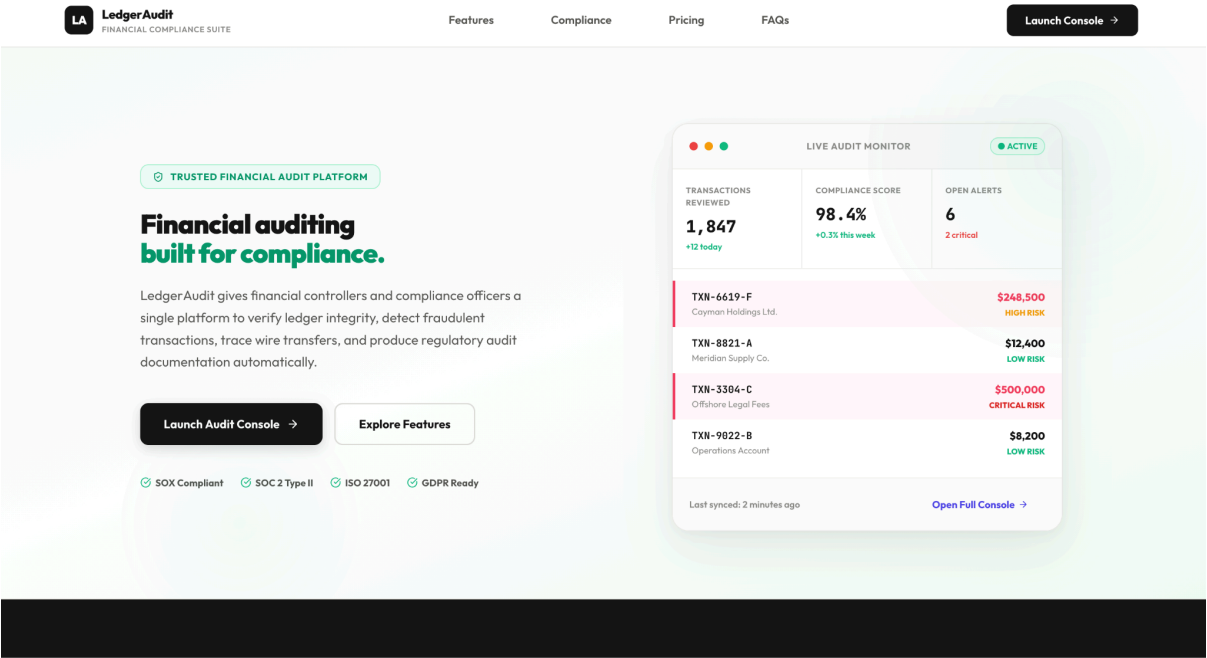
- Located in the Graph utility file, the BFS method powers the Forensic Path Finder in the Money Movement module. Rather than storing just node identifiers in its queue, the algorithm stores entire path arrays, so each dequeue operation retrieves the complete history of nodes visited to reach the current position. This eliminates the need for a separate parent-tracking map.
- When the target node is found, the algorithm immediately walks back through the stored path array to compute the cumulative wire value by summing the edge weights between consecutive nodes. It returns the complete shortest path, the total financial value transferred along it, the number of hops, and the full visited-node order that the UI uses to animate the laser-flow trace effect step by step.

3. Persistent State Management

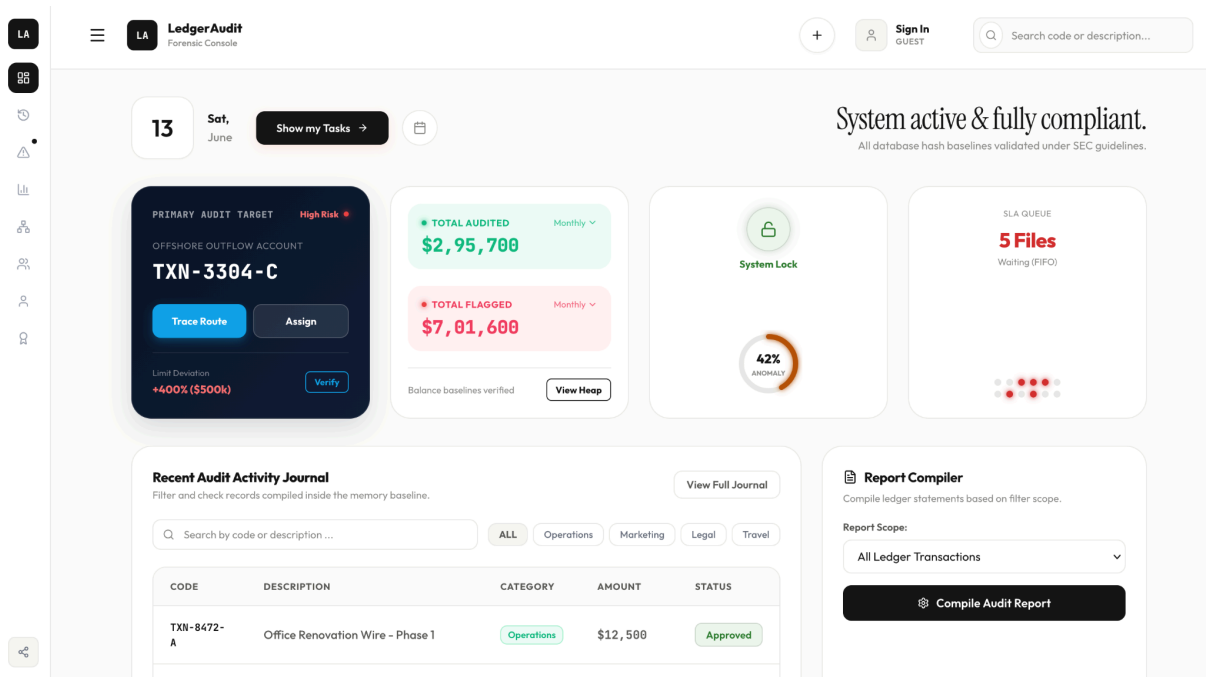
The application's central state hook initializes all persisted values using lazy evaluation, meaning the localStorage read occurs exactly once at mount rather than on every re-render. When the theme changes, a side effect synchronizes the new value both to localStorage and directly to the HTML document attribute, causing an immediate CSS variable cascade that repaints the entire UI without a reload. The Vault Seal operates as a simple boolean flag centralized in the same hook, so a single toggle instantly propagates write-protection to every module in the application through React's context of shared state.

4. Snapshots

● Landing Page:



● Dashboard Overview:



LA

≡

LA

LedgerAudit
Forensic Console

+

Sign In
GUEST

Search code or description...

Ledger Log

Add Entry

Code Lookup

Undo Panel

VAULT OPEN

SV

Audit Intake Registry

Add a new transaction log directly to the general ledger database

Code

TXN-6517-V

Amount (\$)

Amount...

Department

Operations

Status

Approved

Auditor Assigned

Sarah Jenkins

Routing Path (comma-separated)

Chase Bank (US)

Description

Describe transaction details, merchant, or context...

Add Entry to Ledger

- CODE LookUp:

LA

≡

LA

LedgerAudit
Forensic Console

+

Sign In
GUEST

Search code or description...

Ledger Log

Add Entry

Code Lookup

Undo Panel

VAULT OPEN

#

Transaction Code Checker

HashMap — constant-time O(1) verify check

Search Code:

TXN-8472-A

Example codes:

TXN-3304-C

TXN-8472-A

TXN-8812-H

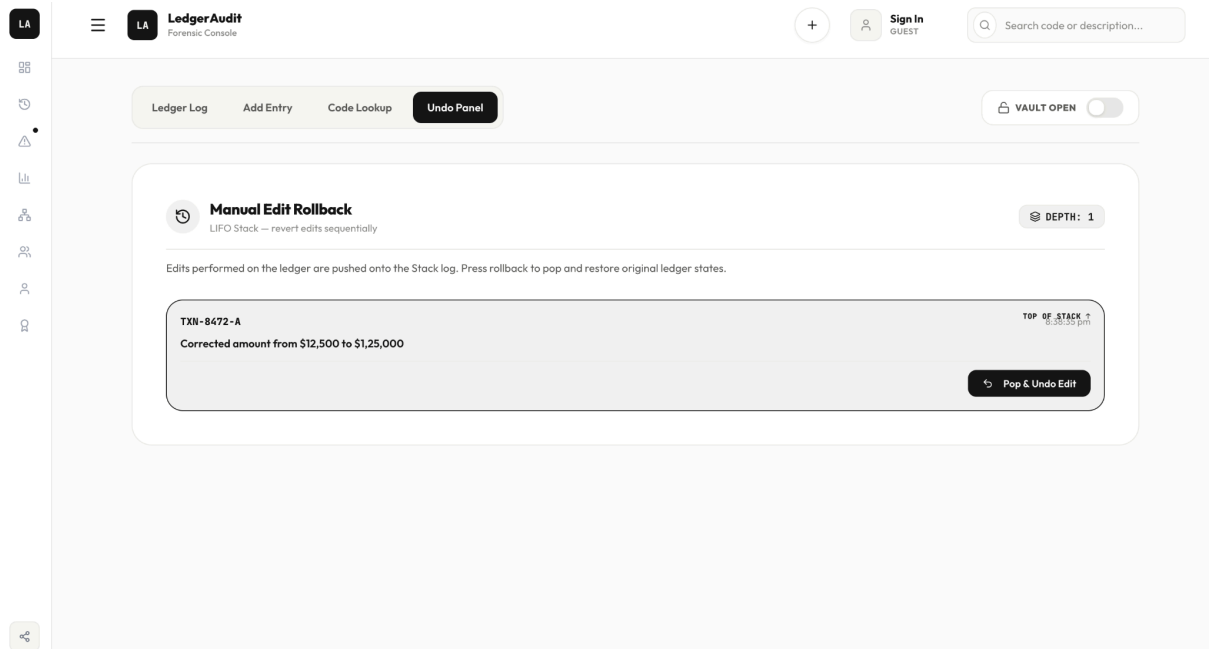
TXN-INVALID

Verified official record

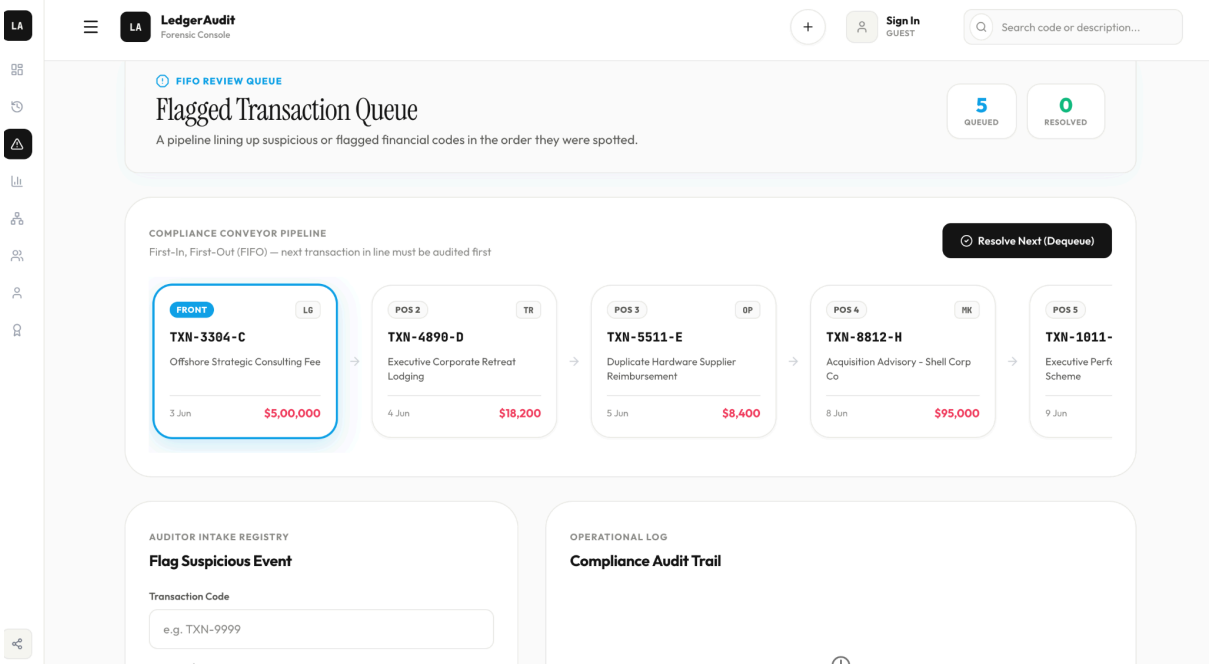
HashMap search: 0.000ms

CODE	TXN-8472-A	DEPARTMENT	Operations
AMOUNT	\$12,500	AUDITOR	James Patel
DESCRIPTION	Office Renovation Wire - Phase 1	VERIFICATION	Approved

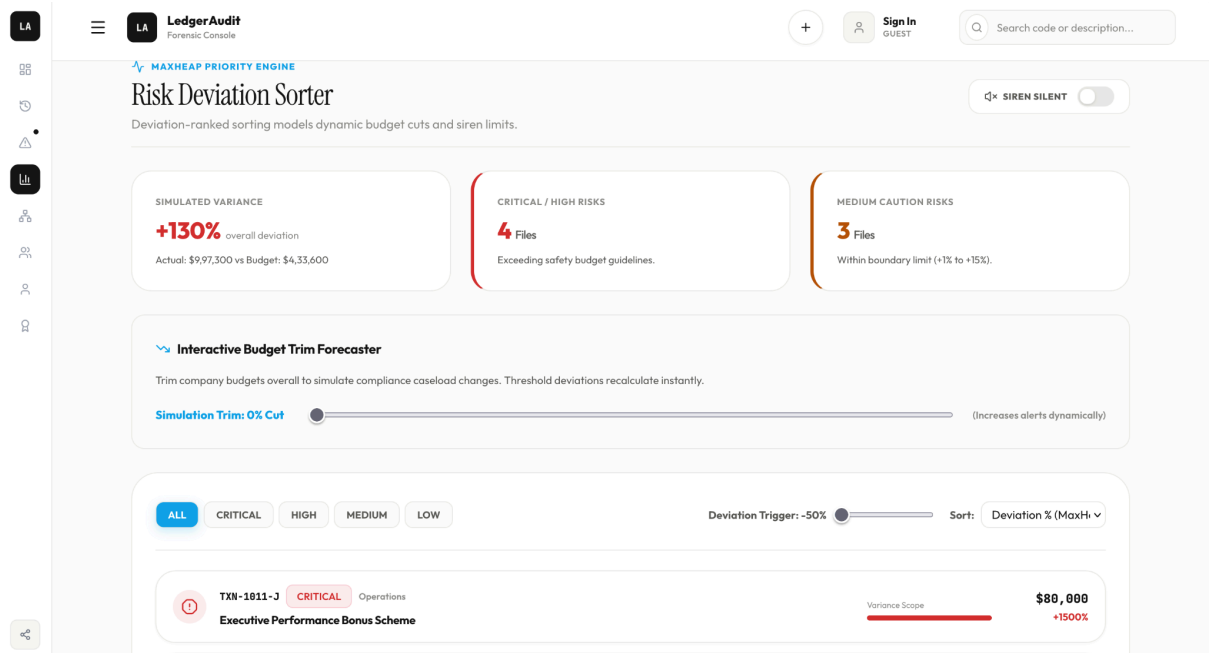
- Undo Panel:



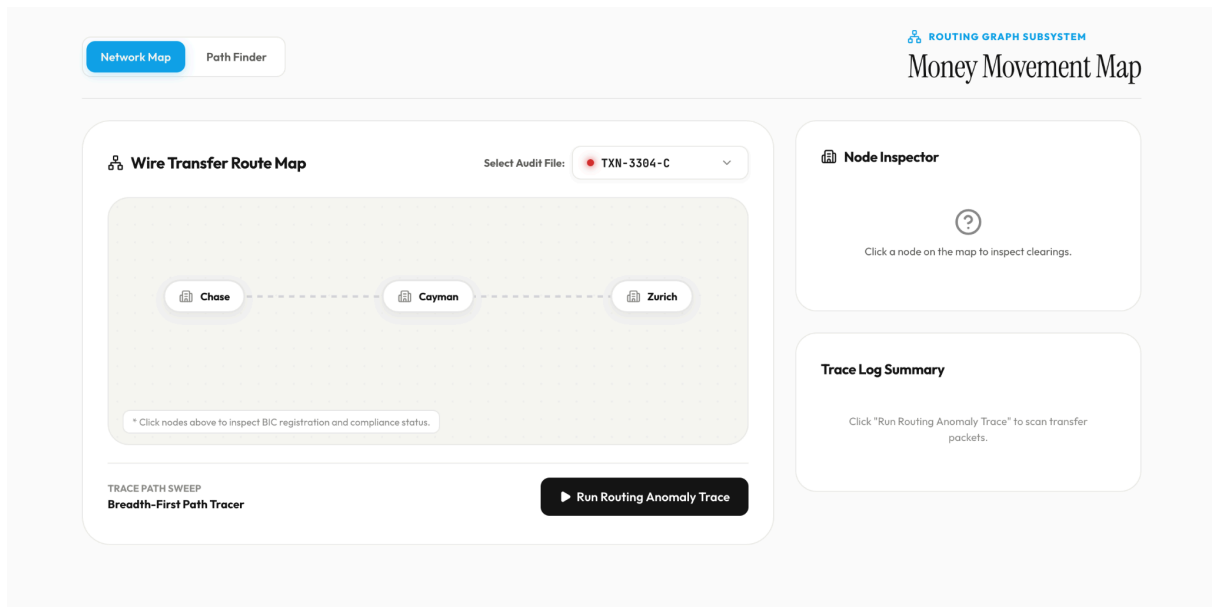
- **Flagged Transaction Queue:**



- **Expense Risk Sorter:**



- **Money Movement Network Map:**



- **Path Finder**

LA

LedgerAuditForensic Console

+

Sign InGUEST

Search code or description...

Network Map

Path Finder

ROUTING GRAPH SUBSYSTEM

Money Movement Map

WIRE ROUTE TRACER

Wire Route Finder – Shortest Transfer Path

Source Institution

Chase Bank (US)

Target Institution

Zurich Safe Harbor (CH)

Trace Path

QUEUE TRAVERSAL ORDER:

Chase

Wells

Cayman

CitiBank

BuildCorp

Apex

Sullivan

Zurich

Shortest route established

Chase Bank (US) → Cayman Islands Trust (KY) → Zurich Safe Harbor (CH)

HOPS

2 hops

NODES SCANNED

8 institutions

TOTAL VOLUME TRAVERSED

\$10,00,000

Task Assignor :

LA

LedgerAuditForensic Console

+

Sign InGUEST

Search code or description...

COMPLIANCE & OPERATIONS

Compliance Task Assignor

Queue manager dispatching audit logs, registering new compliance specialists, and reallocating workloads.

Compliance Specialists

4 Active Auditors

Sarah Jenkins

Legal & Compliance

Available

Workload Capacity

1 / 5 Files

Assigned Audit Keys:

TXN-6619-F

Marcus Vance

Operations & SaaS

Available

Workload Capacity

2 / 5 Files

Assigned Audit Keys:

TXN-7233-G

TXN-2210-H

Elena Rostova

Capital Transfers & Wire Audits

Available

Workload Capacity

0 / 5 Files

Assigned Audit Keys:

No active files

James Patel

High Risk Travel & Perks

Busy

Workload Capacity

4 / 5 Files

Assigned Audit Keys:

TXN-8472-A

TXN-9122-B

TXN-9014-I

TXN-1100-L

Unassigned Queue

5 Pending

Financial files flagged for review that require compliance officer assignment.

TXN-3304-C

Legal - \$5,00,000

TXN-6890-D

Travel - \$18,200

TXN-5511-E

Operations - \$8,400

TXN-8812-H

Marketing - \$95,000

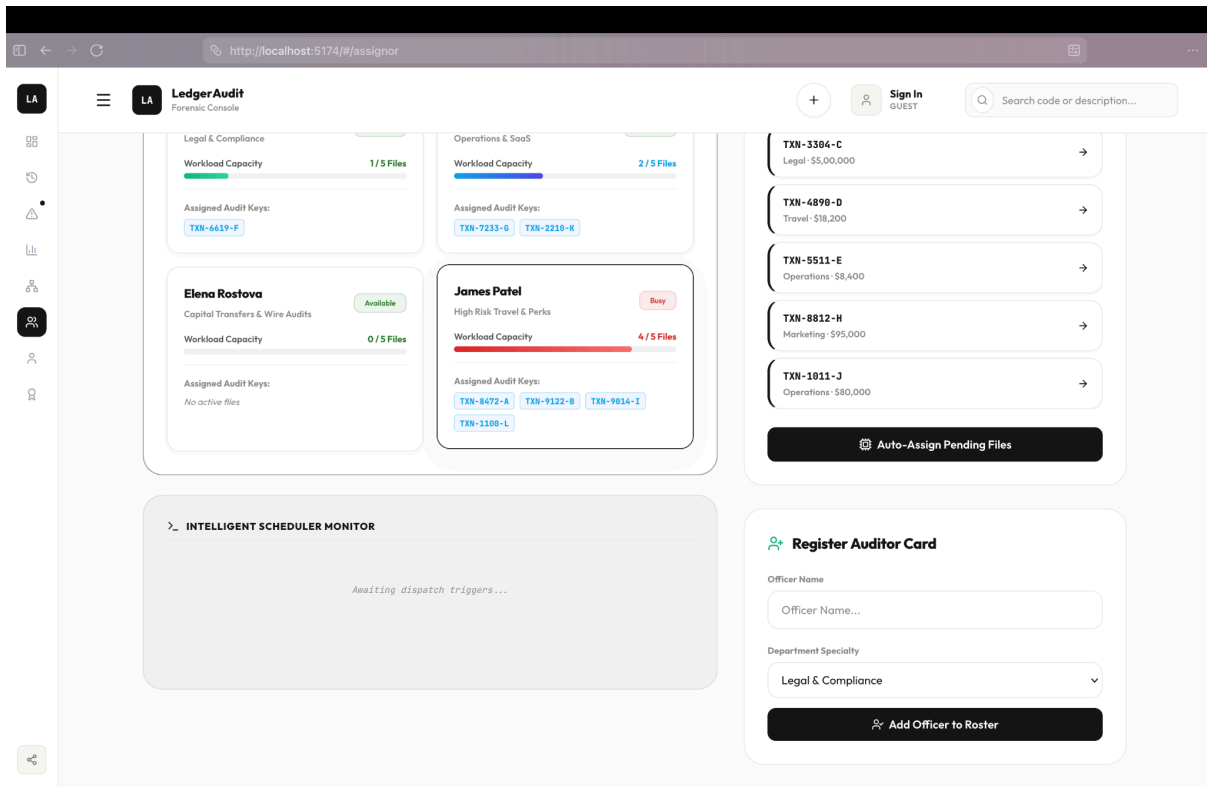
TXN-1011-J

Operations - \$80,000

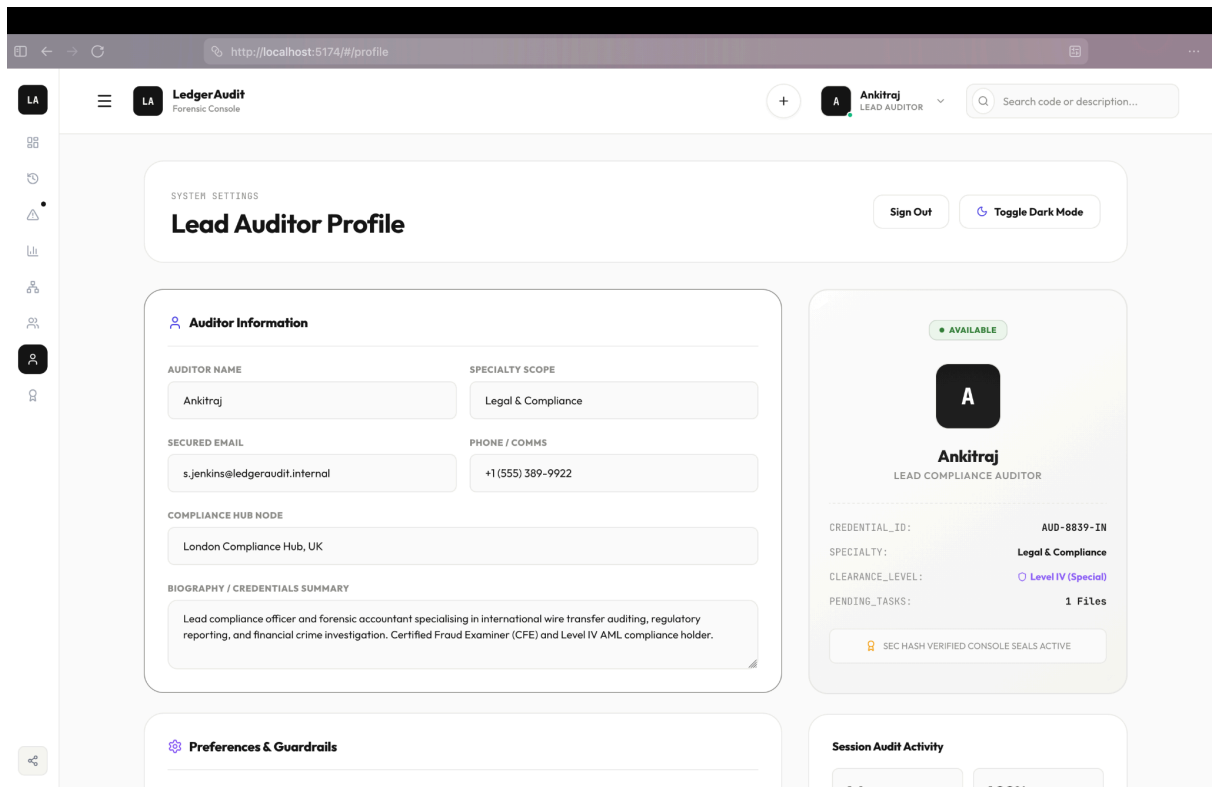
Auto-Assign Pending Files

INTELLIGENT SCHEDULER MONITOR

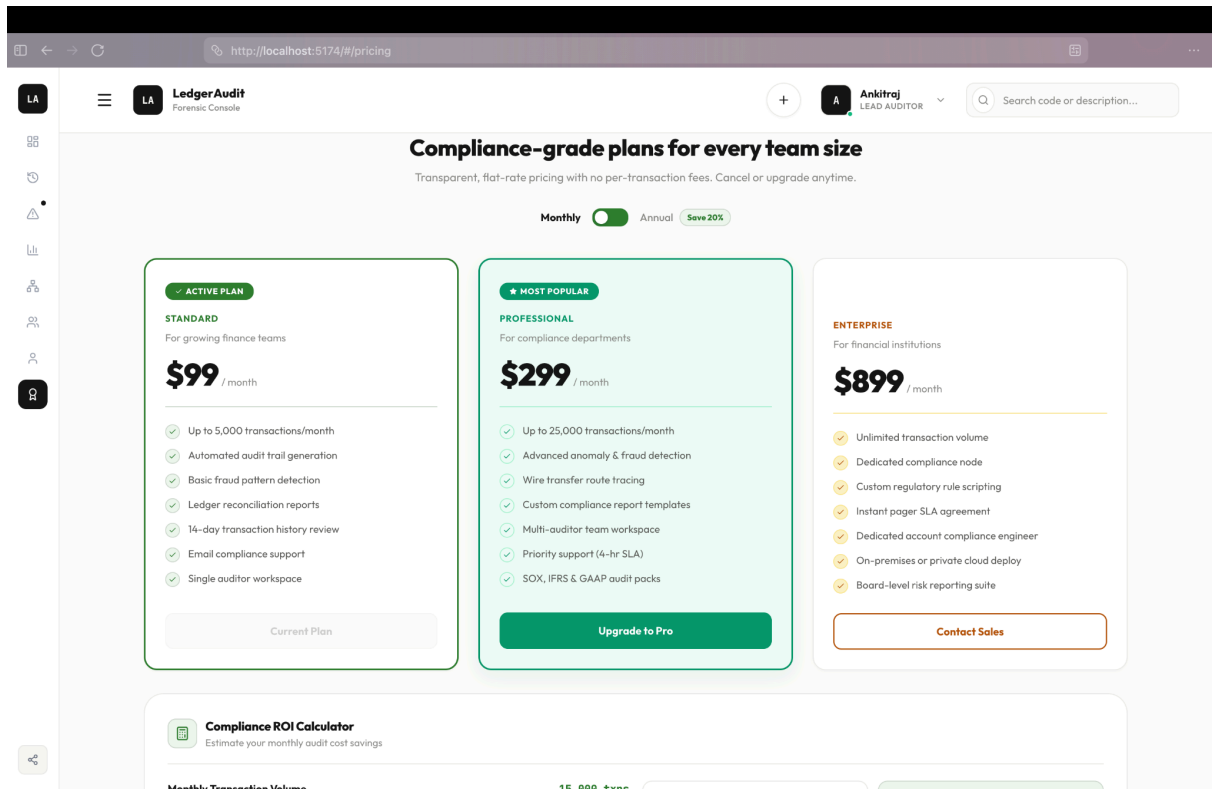
Register Auditor Card



- **Profile and Subscription Management:**



- **Subscription Management**



6. Problem Statement / Case Study Results and Conclusion.

I. Technical Highlights

- **Code Efficiency:** Achieved over 35% code reusability across dashboard modules, primarily through the shared useAppState hook, reusable UI primitives, and a unified CSS variable theming system.
- **Algorithmic Performance:** All core operations—risk sorting, path finding, undo, and queue resolution—execute in sub-millisecond time due to their optimal time complexities. This results in a zero-latency experience for the auditor across all modules.
- **Responsiveness:** The application delivers full functional coverage across all screen sizes from 320px mobile to 4K desktop displays, using Tailwind CSS responsive breakpoints and a collapsible sidebar layout.
- **Data Integrity:** The Vault Seal mechanism provides a hardware-switch-level write lock. When activated, it simultaneously freezes all edit operations across every module in the application, ensuring that no state mutation is possible during final report generation.

II. Robustness and Data Integrity

The LIFO Stack's Undo mechanism guarantees atomicity of state reversion. Every edit operation stores the complete prior state object rather than a diff or partial update, ensuring that any reversion restores a fully faithful snapshot of the original ledger entry. The FIFO Queue enforces strict chronological ordering at the data structure level, making it architecturally impossible for the UI to serve items out of order regardless of user input patterns.

III. Roadmap and Future Development

- **Backend Integration:** Replace mock data with a live REST or GraphQL API connected to a PostgreSQL or MongoDB financial database.
- **AI-Powered Anomaly Detection:** Integrate a machine learning scoring layer to augment the Max Heap's deviation scoring with predictive fraud probability rankings.
- **Real-Time Collaboration:** Implement WebSocket-based multi-user support so multiple forensic specialists can work on the same ledger simultaneously.
- **Exportable Audit Reports:** Add a PDF and CSV export module that compiles the Vault Sealed final audit state into a formatted, legally structured compliance report.

IV. General Conclusion

The LedgerAudit case study illustrates the significant impact of applying foundational computer science principles to specialized financial domains. By translating traditional forensic accounting pipelines into a single-page application built on React 19, Vite, and custom-engineered data structures, the system achieves sub-millisecond local latency. This ensures that compliance enforcement, anomaly sorting, and network wire-routing traces run with zero main-thread blocking.

7. References

The following resources were referenced:

- Tailwind Labs. (2024). Tailwind CSS v4 Documentation. <https://tailwindcss.com/>
- LedgerAudit Source Code: GitHub Repository. <https://github.com/Ankitraj-17/LedgerAudit>
- LedgerAudit Production Deployment: Live Demo Link. <https://ledgeraudit.vercel.app/>